Lab 5.3: Using the Calendar Control

In this lab, you will use the **Calendar** control in your MFC-based application.

To see a demonstration of the lab solution, click this icon.
(CD-ROM plays the demonstration, "Lab 5.3 Demonstration.")

Estimated time to complete this lab: **20 minutes**

To complete the exercises in this lab, you must have the required software. For detailed information about the labs and setup for the labs, see Labs in this course.

The solution code for this lab is located in the folder \Labs\Ch05\Lab5.3\Ex01\Solution.

Objectives

After completing this lab, you will be able to:

- Create an MFC-based application that is a control container.
- Use the Developer Studio Gallery to add the **Calendar** control to your project.
- Use the **Calendar** control in your application.

Prerequisites

There are no prerequisites for this lab.

Exercises

The following exercise provides practice working with the concepts and techniques covered in this chapter:

Exercise 1: Using the Calendar Control

In this exercise, you will create a simple control container application and use the Microsoft® Developer Studio Gallery to add the **Calendar** control to the project. You will include the **Calendar** control in your application and invoke a method and set properties of the control.

160 Exercise 1: Using the Calendar Control

In this exercise, you will create a simple control container application and use the Microsoft Developer Studio Gallery to add the **Calendar** control to the project. You will include the **Calendar** control in your application and invoke a method and set properties of the control.

▶ **Create a control container application**

☐ 1. Start Microsoft Developer Studio and then on the **File** menu, click **New**.

☐ 2. In the **New** dialog box, click the **Projects** tab, and select **MFC AppWizard (exe)**.

☐ 3. In the Location field, type the directory path in which to save the files, set Project Name to **UseCal**, accept the default option, create new workspace, then click **OK**.

☐ 4. In Step 1 of MFC AppWizard, set the type of application to create to **Single Document**, then click **Next**.

☐ 5. In Step 2, accept the default settings and click **Next**.

☐ 6. In Step 3, accept the default of None for compound document support and under other support, make sure that the **ActiveX Controls** check box is selected.

☐ 7. In Step 4, clear the **Printing and print preview** check box.

☐ 8. In Step 5, accept the default options.

☐ 9. In Step 6, derive **CUseCalView** from **CFormView**, rather than **CView,** and click **Finish**.

▶ **Add the Calendar control to your project**

☐ 1. On the **Project** menu, click **Add To Project**, and then click **Components and Controls**.

In the **Components and Controls Gallery** dialog box, double-click the Registered ActiveX Controls folder to see the controls currently registered.

☐ 2. Select **Calendar Control 8.0**, and click **Insert**.

☐ 3. In the **Confirm Classes** dialog box, click **OK**.

☐ 4. In the **Components and Controls Gallery** dialog box, click **Close**.

The **CCalendar** and **COleFont** classes are added to the list of classes in the UseCal application.

☐ 5. In the Developer Studio ClassView pane, expand the **CCalendar** class to view properties and methods of the class.

▶ **Add the Calendar control to your application**

☐ 1. In the ResourceView pane of your project workspace, open the IDD_USECAL_FORM dialog box resource.

☐ 2. Delete the Static **TODO** control and add a **Calendar** control to the dialog box.

☐ 3. Size and position the **Calendar** control appropriately.

☐ 4. Change the ID of the control to IDC_CALENDARCTRL.

Notes

☐ 5. You need to add a member variable for the **Calendar** control to the class that contains the dialog box. In this case, add a member variable to your application's view class as follows:

a. Invoke ClassWizard.

b. On the **Member Variables** tab, click **Add Variable**.

c. In the **Add Member Variable** dialog box, type **m_CalControl** in the **Member variable name** text box and click **OK**.

d. Click **OK** to close the ClassWizard dialog box.

▶ **Add a Calendar menu to your application**

☐ 1. Open the IDR_MAINFRAME menu resource.

☐ 2. In the menu editor, click the empty placeholder in the menu bar and drag it to the position between the **View** and **Help** items.

☐ 3. Double-click the dotted box to display the **Menu Item Properties** property sheet.

Set the **Caption** to **Calendar**.

☐ 4. Add three menu items to the **Calendar** menu with the captions **Show Current Date**, **Set Back Color**, and **Toggle Title**, respectively.

☐ 5. Close the **Menu Item Properties** property sheet.

▶ **Add handlers for the menu items**

☐ 1. Invoke ClassWizard.

☐ 2. On the **Message Maps** tab, select CUseCalView as the class name, ID_CALENDAR_SHOWCURRENTDATE as the object ID, and COMMAND as the message. Create a new function and accept the default **OnCalendarShowcurrentdate** as the function name.

☐ 3. Repeat this procedure to add the default handlers for the ID_CALENDAR_SETBACKCOLOR and ID_CALENDAR_TOGGLETITLE messages.

▶ **Invoke a method and set properties of the Calendar control**

☐ 1. In the **OnCalendarShowcurrentdate** function of the file UseCalView.cpp, add code to invoke the **Calendar** control's **Today** method as follows:

```
m_CalControl.Today();
```

Notes

☐ 2. In the **OnCalendarSetbackcolor** function, add the following code to change the background color of the **Calendar** control:

```
OLE_COLOR current_color = m_CalControl.GetBackColor();
COLORREF old_color;
// OLE_COLOR must be converted to a COLORREF for use
// by CColorDialog.
if (S_OK != ::OleTranslateColor(current_color,
                                NULL, &old_color))
    return;

CColorDialog dlg(old_color);

if (IDOK == dlg.DoModal())
{
    // No translation is needed from COLORREF to OLE_COLOR
    COLORREF new_color = dlg.GetColor();
    if (new_color != old_color)
    {
        current_color = new_color;
        m_CalControl.SetBackColor(current_color);
    }
    }
```

☐ 3. In the **OnCalendarToggleTitle** function, add the following code to toggle the display of the title on the **Calendar** control:

```
BOOL bTitleState = m_CalControl.GetShowTitle();
m_CalControl.SetShowTitle(! bTitleState);
```

▶ **Add an event handler for the Calendar control**

In this procedure, you will add and implement a handler for the double-click event to enable the user to set the system date.

☐ 1. Invoke ClassWizard.

☐ 2. On the **Message Maps** tab, select CUseCalView as the Class name, IDC_CALENDARCTRL as the Object ID and DbClick as the Message.

☐ 3. Click **Add Function** and accept the default name of **OnDblClickCalendarctrl**.

ClassWizard will insert an event sink map in the class with an entry for the double-click event.

Notes

☐ 4. Write the code for the **OnDblClickCalendarctrl** function to change the system date as follows:

```
short year = m_CalControl.GetYear();
short month = m_CalControl.GetMonth();
short day = m_CalControl.GetDay();
WORD hh,min,sec,mlsec;
SYSTEMTIME mtime;

GetSystemTime(&mtime);
hh=mtime.wHour;
min=mtime.wMinute;
sec=mtime.wSecond;
mlsec=mtime.wMilliseconds;

mtime.wYear= (WORD)year;
mtime.wMonth= (WORD)month;
mtime.wDay= (WORD)day;
mtime.wHour=hh;
mtime.wMinute=min;
mtime.wSecond=sec;
mtime.wMilliseconds=mlsec;

CString str;
str.Format("Set system date to: %d/%d/%d ?",month,day,year);

int i=::MessageBox(0,str,"Date",MB_YESNO);
if(i==IDYES)
    ::SetSystemTime(&mtime);
```

▶ **Build and test your application**

- Run your application and test the options under the **Calendar** menu. Double-click a date on the control and try changing the system date.

The solution code for this exercise is located in the folder \Labs\Ch05\Lab5.3\Ex01\Solution.

Notes